

Buscar un archivo específico dentro de un sistema de archivos

El código implementa un programa en Python que busca un archivo específico (por su nombre) dentro de un sistema de archivos, partiendo de un directorio inicial proporcionado por el usuario. Utiliza algoritmos de búsqueda no informada: **búsqueda en profundidad (DFS)** y **búsqueda en amplitud (BFS)**, representando el sistema de archivos como un grafo donde los nodos son directorios o archivos, y las aristas son las conexiones entre ellos (es decir, los elementos dentro de un directorio). La solución, si se encuentra, es la ruta completa desde el directorio inicial hasta el archivo objetivo, presentada como una secuencia de nombres de directorios y el archivo final.

Estructura general del código

El código está organizado en tres clases principales y un bloque de ejecución principal:

1. **Clase `Node`**: Representa un nodo en el grafo de búsqueda, que corresponde a un directorio o archivo en el sistema de archivos.
2. **Clase `StackFrontier`**: Implementa una frontera basada en una pila (LIFO) para la búsqueda en profundidad (DFS).
3. **Clase `QueueFrontier`**: Extiende `StackFrontier` para implementar una frontera basada en una cola (FIFO) para la búsqueda en amplitud (BFS).
4. **Clase `FileSearch`**: Gestiona la lógica de búsqueda en el sistema de archivos, incluyendo la inicialización, exploración de directorios y presentación de resultados.
5. **Bloque principal (`if __name__ == "__main__"`)**: Maneja los argumentos de entrada y ejecuta la búsqueda con DFS y BFS.

Componentes

1. Clase Node

```
class Node:
    def __init__(self, state, parent, action):
        self.state = state    # Ruta completa del directorio o archivo actual
        self.parent = parent  # Nodo padre
        self.action = action  # Nombre del directorio o archivo al que se accede
```

Propósito: Representa un nodo en el grafo de búsqueda, que corresponde a un directorio o archivo en el sistema de archivos.

Atributos:

- **state:** La ruta absoluta del directorio o archivo actual (por ejemplo, `/home/user/docs`).
- **parent:** El nodo desde el cual se generó este nodo (o `None` si es el nodo inicial).
- **action:** El nombre del directorio o archivo que lleva al estado actual desde el nodo padre (por ejemplo, `docs` o `file.txt`).
- **Uso:** Permite rastrear la secuencia de pasos (acciones) desde el directorio inicial hasta el archivo objetivo, reconstruyendo la ruta completa al encontrar la solución.

2. Clase StackFrontier

```
class StackFrontier:
    def __init__(self):
        self.frontier = []

    def add(self, node):
        self.frontier.append(node)

    def contains_state(self, state):
        return any(node.state == state for node in self.frontier)

    def empty(self):
        return len(self.frontier) == 0
```

```
def remove(self):
    if self.empty():
        raise Exception("empty frontier")
    else:
        node = self.frontier[-1]
        self.frontier = self.frontier[:-1]
        return node
```

Propósito: Gestiona la frontera de nodos pendientes de exploración usando una pila (LIFO), lo que implementa la búsqueda en profundidad (DFS).

Métodos:

- `__init__`: Inicializa una lista vacía para almacenar los nodos de la frontera.
- `add(node)`: Añade un nodo a la cima de la pila.
- `contains_state(state)`: Verifica si un estado (ruta) ya está en la frontera.
- `empty()`: Comprueba si la frontera está vacía.
- `remove()`: Extrae y devuelve el nodo más reciente (último en la pila), eliminándolo de la frontera.

Uso en DFS: La pila asegura que el algoritmo explore primero los nodos más profundos, avanzando por una rama del sistema de archivos antes de retroceder.

3. Clase QueueFrontier

```
class QueueFrontier(StackFrontier):
    def remove(self):
        if self.empty():
            raise Exception("empty frontier")
        else:
            node = self.frontier[0]
            self.frontier = self.frontier[1:]
            return node
```

Propósito: Extiende `StackFrontier` para implementar una cola (FIFO), usada en la búsqueda en amplitud (BFS).

Método sobrescrito:

- `remove()`: Extrae y devuelve el nodo más antiguo (primero en la cola), eliminándolo de la frontera.

Uso en BFS: La cola asegura que los nodos se exploren en el orden en que fueron generados, priorizando los niveles más cercanos al directorio inicial.

4. Clase FileSearch

```
class FileSearch:
    def __init__(self, start_dir, target_file):
        # Validar que el directorio inicial exista
        if not os.path.isdir(start_dir):
            raise Exception("start_dir must be a valid directory")
        self.start = os.path.abspath(start_dir) # Ruta absoluta del directorio inicial
        self.target = target_file # Nombre del archivo objetivo
        self.solution = None
        self.num_explored = 0
        self.explored = set()
```

Propósito: Define la lógica para buscar un archivo en el sistema de archivos.

Atributos:

- `start`: Ruta absoluta del directorio inicial (por ejemplo, `/home/user`).
- `target`: Nombre del archivo a buscar (por ejemplo, `documento.txt`).
- `solution`: Almacena la solución encontrada (acciones y estados) o `None` si no se encuentra.
- `num_explored`: Contador de directorios explorados.
- `explored`: Conjunto de rutas ya visitadas para evitar duplicados.

Inicialización: Valida que el directorio inicial exista y convierte su ruta a absoluta usando `os.path.abspath`.

```
def print(self):
    """Imprime el estado inicial, la solución (si existe) y los directorios explorados."""
    print(f"\nStarting directory: {self.start}")
    if self.solution is not None:
        print("Solution found:")
```

```

    print("Path:", " -> ".join(self.solution[1]))
else:
    print("No solution found.")
print(f"Directories explored: {self.num_explored}")
#print("Explored directories:", sorted(list(self.explored)))
print("Explored directories:")
count = 1
for node in sorted(list(self.explored)):
    print(f"{count}: {node}")
    count += 1

```

Propósito: Muestra el estado inicial, la solución (si existe) y los directorios explorados.

Funcionamiento: Imprime la ruta inicial, la secuencia de directorios/archivos que lleva al objetivo (si se encontró), el número de directorios explorados y una lista ordenada de todas las rutas visitadas.

```

def neighbors(self, state):
    """Devuelve los directorios y archivos accesibles desde el estado actual
    (directorio)."""
    result = []
    try:
        # Listar contenido del directorio actual
        for item in os.listdir(state):
            item_path = os.path.join(state, item)
            # Incluir solo directorios y archivos accesibles
            if os.path.isdir(item_path) or os.path.isfile(item_path):
                result.append((item, item_path))
    except (PermissionError, OSError):
        # Ignorar directorios o archivos a los que no se tiene acceso
        pass
    return result

```

Propósito: Genera los "vecinos" del estado actual, es decir, los directorios y archivos accesibles dentro del directorio actual.

Funcionamiento:

- Usa `os.listdir` para listar el contenido del directorio (`state`).
- Crea rutas completas con `os.path.join`.

- Incluye solo directorios y archivos válidos, ignorando otros tipos de elementos.
- Maneja excepciones como `PermissionError` para evitar errores en directorios restringidos.

Salida: Lista de tuplas (`action`, `state`), donde `action` es el nombre del elemento (por ejemplo, `docs`) y `state` es la ruta completa (por ejemplo, `/home/user/docs`).

```
def solve(self, use_bfs=False):
    """Encuentra un archivo objetivo en el sistema de archivos usando DFS o BFS."""
    # Inicializar la frontera con el directorio inicial
    start = Node(state=self.start, parent=None, action=None)
    frontier = QueueFrontier() if use_bfs else StackFrontier()
    frontier.add(start)
    self.explored = set()

    while True:
        # Si la frontera está vacía, no hay solución
        if frontier.empty():
            raise Exception("no solution")

        # Seleccionar un nodo de la frontera
        node = frontier.remove()
        self.num_explored += 1

        # Verificar si el nodo es el archivo objetivo
        if os.path.isfile(node.state) and os.path.basename(node.state) == self.target:
            actions = []
            paths = []
            while node.parent is not None:
                actions.append(node.action)
                paths.append(node.state)
                node = node.parent
            actions.reverse()
            paths.reverse()
            self.solution = (actions, paths)
            return

        # Marcar el nodo como explorado
        self.explored.add(node.state)

        # Agregar vecinos (directorios y archivos) a la frontera
        for action, state in self.neighbors(node.state):
            if not frontier.contains_state(state) and state not in self.explored:
```

```
child = Node(state=state, parent=node, action=action)
frontier.add(child)
```

Propósito: Implementa el algoritmo de búsqueda para encontrar el archivo objetivo.

Parámetros:

- `use_bfs`: Si es `True`, usa BFS (`QueueFrontier`); si es `False`, usa DFS (`StackFrontier`).

Funcionamiento:

1. Inicializa la frontera con el nodo correspondiente al directorio inicial.
2. Mantiene un conjunto `explored` para evitar visitar el mismo directorio o archivo dos veces.
3. Itera hasta encontrar la solución o agotar la frontera:
 - Si la frontera está vacía, lanza una excepción indicando que no hay solución.
 - Extrae un nodo de la frontera y lo marca como explorado.
 - Verifica si el nodo es un archivo cuyo nombre coincide con `target` usando `os.path.isfile` y `os.path.basename`.
 - Si es el objetivo, reconstruye la solución (acciones y rutas) retrocediendo desde el nodo actual hasta el inicial.
 - Si no es el objetivo, genera los vecinos del nodo y los añade a la frontera si no han sido explorados ni están en la frontera.

Solución: Almacena una tupla `(actions, paths)`, donde `actions` son los nombres de los directorios/archivos en la ruta, y `paths` son las rutas absolutas correspondientes.

5. Bloque Principal

```
if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit("Usage: python file_search.py start_directory target_file")

    start_dir = sys.argv[1]
    target_file = sys.argv[2]
```

```

print(f"Searching for file: {target_file}")
fs = FileSearch(start_dir, target_file)

print("Solving with DFS...")
try:
    fs.solve(use_bfs=False)
    fs.print()
except Exception as e:
    print(f"Error: {e}")

# Opcionalmente, resolver con BFS
print("\nSolving with BFS...")
fs = FileSearch(start_dir, target_file)
try:
    fs.solve(use_bfs=True)
    fs.print()
except Exception as e:
    print(f"Error: {e}")

```

Propósito: Ejecuta el programa, tomando los argumentos de línea de comandos y corriendo ambas búsquedas (DFS y BFS).

Entrada:

- `sys.argv[1]`: Directorio inicial (por ejemplo, `/home/user`).
- `sys.argv[2]`: Nombre del archivo objetivo (por ejemplo, `documento.txt`).

Funcionamiento:

- Valida que se proporcionen exactamente dos argumentos.
- Crea una instancia de `FileSearch` con el directorio inicial y el archivo objetivo.
- Ejecuta la búsqueda con DFS, imprime los resultados o maneja errores.
- Repite el proceso con BFS para comparar resultados.

Salida: Muestra el archivo buscado, el directorio inicial, la ruta al archivo (si se encuentra), el número de directorios explorados y la lista de directorios visitados.

Cómo funciona el algoritmo

El programa modela el sistema de archivos como un grafo donde:

- **Nodos** son directorios o archivos (estados).
- **Aristas** son las conexiones entre un directorio y sus contenidos (acciones).
- **Estado inicial** es el directorio inicial.
- **Estado objetivo** es el archivo cuyo nombre coincide con `target_file`.

El algoritmo explora el sistema de archivos iterativamente:

1. Comienza en el directorio inicial, añadiéndolo a la frontera.
2. Extrae un nodo de la frontera:
 - En **DFS**, toma el nodo más reciente (pila), explorando primero los directorios más profundos.
 - En **BFS**, toma el nodo más antiguo (cola), explorando primero los directorios más cercanos al inicial.
3. Verifica si el nodo es el archivo objetivo.
4. Si no lo es, marca el nodo como explorado y añade sus vecinos (archivos y subdirectorios) a la frontera, evitando duplicados.
5. Repite hasta encontrar el archivo o agotar la frontera.

Aspectos adicionales relevantes

1. Manejo de errores:

- Valida que el directorio inicial sea válido (`os.path.isdir`).
- Ignora errores de permisos o acceso (`PermissionError`, `OSError`) al listar directorios, asegurando robustez en sistemas con restricciones.

2. Eficiencia:

- **DFS** es adecuada para sistemas de archivos profundos, ya que usa menos memoria al explorar una rama a la vez, pero puede tardar más si el archivo está en una rama diferente.
- **BFS** garantiza encontrar el archivo en el menor número de pasos (en términos de niveles de directorios), pero consume más memoria al almacenar todos los nodos de un nivel.

3. Limitaciones:

- No maneja enlaces simbólicos o ciclos en el sistema de archivos, lo que podría causar bucles infinitos si no se controla.
- La búsqueda puede ser lenta en sistemas de archivos grandes, especialmente con BFS, debido al número de nodos en la frontera.
- No implementa heurísticas (búsqueda informada), lo que podría optimizar la búsqueda en sistemas de archivos grandes.

4. Posibles mejoras:

- Añadir soporte para buscar por patrones (por ejemplo, `*.txt`) usando `fnmatch`.
- Implementar búsqueda informada (como A*) usando heurísticas, como priorizar directorios con nombres similares al archivo objetivo.
- Agregar un límite de profundidad para DFS o un límite de nodos explorados para evitar búsquedas prolongadas.

Ejemplo de uso

Para este ejercicio, se construyó la siguiente estructura de directorios y archivos (puede utilizar cualquier otra estructura):

```
$ tree test/
test/
├── a
│   ├── a1
│   │   ├── a11.txt
│   │   ├── a12.txt
│   │   └── a13.txt
│   ├── a2
│   │   ├── a21.txt
│   │   ├── a22.txt
│   │   └── a23.txt
│   └── a3
│       ├── a31.txt
│       ├── a32.txt
│       └── a33.txt
└── b
```

```
| | | | | b1
| | | | | | | | | b11.txt
| | | | | | | | | b12.txt
| | | | | | | | | b13.txt
| | | | | b2
| | | | | | | | | b21.txt
| | | | | | | | | b22.txt
| | | | | | | | | b23.txt
| | | | | b3
| | | | | | | | | b31.txt
| | | | | | | | | b32.txt
| | | | | | | | | b33.txt
| | | | | c
| | | | | | | | | c1
| | | | | | | | | | | c11.txt
| | | | | | | | | | | c12.txt
| | | | | | | | | | | c13.txt
| | | | | | | | | c2
| | | | | | | | | | | c21.txt
| | | | | | | | | | | c22.txt
| | | | | | | | | | | c23.txt
| | | | | | | | | c3
| | | | | | | | | | | c31.txt
| | | | | | | | | | | c32.txt
| | | | | | | | | | | c33.txt
```

13 directories, 27 files

Comando: `python file_search.py test c12.txt`

Salida (ejemplo):

```
$ python file_search.py test/ c12.txt
Searching for file: c12.txt
Solving with DFS...

Starting directory: test
Solution found:
Path: test/c -> test/c/c1 -> test/c/c1/c12.txt
Directories explored: 13
Explored directories:
1: test
2: test/c
```

```
3: test/c/c1
4: test/c/c1/c13.txt
5: test/c/c2
6: test/c/c2/c21.txt
7: test/c/c2/c22.txt
8: test/c/c2/c23.txt
9: test/c/c3
10: test/c/c3/c31.txt
11: test/c/c3/c32.txt
12: test/c/c3/c33.txt
```

Solving with BFS...

Starting directory: test

Solution found:

Path: test/c -> test/c/c1 -> test/c/c1/c12.txt

Directories explored: 33

Explored directories:

```
1: test
2: test/a
3: test/a/a1
4: test/a/a1/a11.txt
5: test/a/a1/a12.txt
6: test/a/a1/a13.txt
7: test/a/a2
8: test/a/a2/a21.txt
9: test/a/a2/a22.txt
10: test/a/a2/a23.txt
11: test/a/a3
12: test/a/a3/a31.txt
13: test/a/a3/a32.txt
14: test/a/a3/a33.txt
15: test/b
16: test/b/b1
17: test/b/b1/b11.txt
18: test/b/b1/b12.txt
19: test/b/b1/b13.txt
20: test/b/b2
21: test/b/b2/b21.txt
22: test/b/b2/b22.txt
23: test/b/b2/b23.txt
24: test/b/b3
25: test/b/b3/b31.txt
26: test/b/b3/b32.txt
27: test/b/b3/b33.txt
28: test/c
29: test/c/c1
30: test/c/c1/c11.txt
31: test/c/c2
32: test/c/c3
```

Comando: `python file_search.py test a12.txt`

Salida (ejemplo):

```
$ python file_search.py test/ a12.txt
Searching for file: a12.txt
Solving with DFS...

Starting directory: test
Solution found:
Path: test/a -> test/a/a1 -> test/a/a1/a12.txt
Directories explored: 39
Explored directories:
1: test
2: test/a
3: test/a/a1
4: test/a/a1/a13.txt
5: test/a/a2
6: test/a/a2/a21.txt
7: test/a/a2/a22.txt
8: test/a/a2/a23.txt
9: test/a/a3
10: test/a/a3/a31.txt
11: test/a/a3/a32.txt
12: test/a/a3/a33.txt
13: test/b
14: test/b/b1
15: test/b/b1/b11.txt
16: test/b/b1/b12.txt
17: test/b/b1/b13.txt
18: test/b/b2
19: test/b/b2/b21.txt
20: test/b/b2/b22.txt
21: test/b/b2/b23.txt
22: test/b/b3
23: test/b/b3/b31.txt
24: test/b/b3/b32.txt
25: test/b/b3/b33.txt
26: test/c
27: test/c/c1
28: test/c/c1/c11.txt
29: test/c/c1/c12.txt
30: test/c/c1/c13.txt
31: test/c/c2
32: test/c/c2/c21.txt
33: test/c/c2/c22.txt
34: test/c/c2/c23.txt
35: test/c/c3
```

```
36: test/c/c3/c31.txt  
37: test/c/c3/c32.txt  
38: test/c/c3/c33.txt
```

Solving with BFS...

Starting directory: test

Solution found:

Path: test/a -> test/a/a1 -> test/a/a1/a12.txt

Directories explored: 15

Explored directories:

```
1: test  
2: test/a  
3: test/a/a1  
4: test/a/a1/a11.txt  
5: test/a/a2  
6: test/a/a3  
7: test/b  
8: test/b/b1  
9: test/b/b2  
10: test/b/b3  
11: test/c  
12: test/c/c1  
13: test/c/c2  
14: test/c/c3
```